

Finance Video Q&A Bot

Project Documentation

A Multimodal AI Chatbot for YouTube Video Q&A

Author: João • Ironhack — AI Engineering Bootcamp • Final Project

1. Project Overview

The Finance Video Q&A Bot is a multimodal AI chatbot that answers natural-language questions about a collection of personal-finance YouTube videos. Users can interact by text or by voice, and every answer is grounded in the actual video content, with a link to the exact moment the topic is discussed. The system combines speech recognition, retrieval-augmented generation (RAG), an LLM agent, and text-to-speech, and is deployed as a public web app.

Live app: finance-ai-bot-finalproject.streamlit.app

2. Objectives

- Build a text-based question-answering system using a pre-trained language model.
- Integrate speech recognition to convert voice input into text.
- Provide a conversational interface supporting both text and voice.
- Retrieve, process and store video content in a vector database.
- Use an agent with multiple tools and conversational memory.
- Test and evaluate the bot's performance.

3. System Architecture

The system is split into two phases. An offline ingestion phase transcribes and indexes the videos once; an online phase serves user questions in real time.

Offline (ingestion)

- YouTube → yt-dlp → faster-whisper → chunks + embeddings → ChromaDB.

Videos are downloaded as audio, transcribed locally with Whisper, split into timestamped chunks, embedded, and stored in the vector database.

Online (live app)

- User question (text or voice) → LangChain agent (GPT-4o-mini) → retrieval tool → ChromaDB → grounded answer + sources (with timestamps) → optional text-to-speech.

Design principle: heavy models (Whisper) run offline for batch transcription, while the live app uses lightweight OpenAI APIs — keeping the deployed server small and responsive.

4. Methodology

4.1 Data ingestion & transcription

- 13 personal-finance YouTube videos covering investing, ETFs, budgeting, emergency funds, loans, dividends and more.
- Audio downloaded with yt-dlp.
- Transcription with faster-whisper (small.en model), with a fallback to existing YouTube captions via youtube-transcript-api when available.
- A 3-second delay between downloads avoids overloading YouTube; cached audio is reused to avoid re-downloading.

4.2 Retrieval-Augmented Generation (RAG)

- Transcripts are split into ~1000-character chunks with 150-character overlap, built by grouping Whisper segments so each chunk keeps its start timestamp.
- Each chunk is embedded with OpenAI text-embedding-3-small.
- Chunks and metadata (video_id, title, channel, url, source, start_time) are stored in ChromaDB (~190 chunks).
- At query time, the most semantically similar chunks are retrieved and passed to the LLM as context.

4.3 Q&A Agent

- A LangChain tool-calling agent (create_tool_calling_agent + AgentExecutor) using GPT-4o-mini.
- The agent decides which tool to call for each question.
- Conversational memory via RunnableWithMessageHistory keeps per-session context, enabling follow-up questions.
- The system prompt enforces grounding (answer only from retrieved context), conciseness, an explicit "I don't know" when context is missing, and a financial-advice disclaimer.

4.4 Voice (multimodal)

- Speech-to-text: the deployed app uses the OpenAI Whisper API to transcribe the user's recorded question.
- Text-to-speech: OpenAI TTS (tts-1) optionally reads the answer aloud.

5. LangChain & LangSmith Usage

- LangChain agents and tools orchestrate the Q&A flow (AgentExecutor, @tool functions).
- LangChain Expression Language (LCEL) builds the RAG chain (retriever | prompt | LLM | parser).
- RunnableWithMessageHistory provides conversational memory.
- LangChain integrations: langchain-openai (LLM + embeddings) and langchain-community (Chroma vector store).
- LangSmith provides tracing of every run and the evaluation framework (datasets + LLM-as-a-judge).

6. Evaluation

The bot was evaluated with LangSmith using an LLM-as-a-judge over a 14-question test set, with two metrics:

- Groundedness: 1.00 — answers are supported by the retrieved context; the bot does not hallucinate.
- Correctness: 0.79 — answers compared against reference answers.

The correctness gap is largely a matter of corpus coverage: when a topic is not in the videos, the bot correctly declines rather than inventing an answer, which the reference-based judge marks as incorrect. The perfect groundedness score confirms the gap is coverage, not fabricated facts. LangSmith also tracks latency and token cost per run; total evaluation cost was a fraction of a cent.

7. Deployment

The app is deployed on Streamlit Community Cloud and is publicly accessible via URL. API keys are managed through the platform's secrets manager. Voice input on the deployed app uses the OpenAI Whisper API (instead of the local model) so the server remains lightweight.

8. Technology Stack

Area	Technology
Audio download	yt-dlp
Transcription (STT)	faster-whisper (small.en); captions fallback (youtube-transcript-api)
Framework	LangChain (agent, tools, memory, LCEL)
Embeddings	OpenAI text-embedding-3-small
Vector database	ChromaDB
LLM	OpenAI gpt-4o-mini
Voice	OpenAI Whisper API (whisper-1) + OpenAI TTS (tts-1)
Interface	Streamlit
Evaluation / tracing	LangSmith (LLM-as-a-judge)
Deployment	Streamlit Community Cloud

9. Limitations & Future Work

- Corpus limited to 13 videos; a larger corpus would broaden coverage.
- English only; multi-language support is a natural extension.
- Inline citations within the answer text (not only in a sources panel).
- A larger, more challenging evaluation set.

10. Disclaimer

This project is an educational and technical demonstration. The bot's answers do not constitute financial advice. For financial decisions, consult a certified professional.